

Table of Contents

Background	1
Processing	1
File Structures and Sizes	1
Schemas	2
Transforming Metadata Tables.....	3
Inventory	3
Enriched Stations Table.....	5
Merging with Daily	5
Analysis	5
Affiliations of stations.....	5
Geographical Distance(s)	6
Daily Observations and Tasks.....	7
Total Observations	7
Core Element Counts.....	8
Temperature Matching	8
Temperatures in New Zealand	9
Average Temperatures in NZ as reported by GHCND:.....	9
Average Rainfall Worldwide	10
Conclusions:.....	10

Background

GHCND records weather and environmental condition data in 219 countries worldwide; specifically, they record the occurrence of weather events and their relative value.

These events are identified by their DATE and the location of the station recording the weather event.

While the daily observations are the core datapoints; there are metadata tables describing the stations, countries, states and breadth of observations in 'inventory'.

As the volume of observations is measurable in billions of rows; to transform, augment or process this in a parallelized way; we employ a big data processing stack. That is, a computing cluster with the Hadoop File System and a Spark python Executor at the highest level.

The core complexity of this data is the amount of it; while smaller datasets could still be handled in memory, in trivial time; in fact, this is the case for all the operations we will run for the metadata tables. This has run on effects where, due to how the data is compressed, some spark methods which would typically save computational overhead (or, rather, waiting time) are of no benefit.

Supporting files and graphs are available in the user directory.

Processing

File Structures and Sizes

The historical weather observation data is given in two formats; /daily contains records in the form of compressed .gz files; organized into sub files by year. Stations, Countries, inventory and states are defined as txt files only and have no compression. For the purpose of analysis, all files and references to the ghcnd original files are scoped to the hdfs cluster under /data. See below:

The file structure for /ghcnd is as follows:

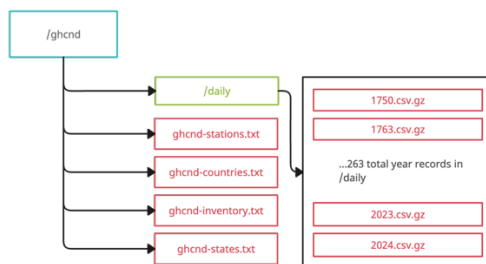


Figure 1 - Directory Structure for /ghcnd

Where /daily contains 263 total year records, distributed from 1750 through to 2024; noting that data there are missing records between 1750 and 1763 and that the records are in compressed .gz form.

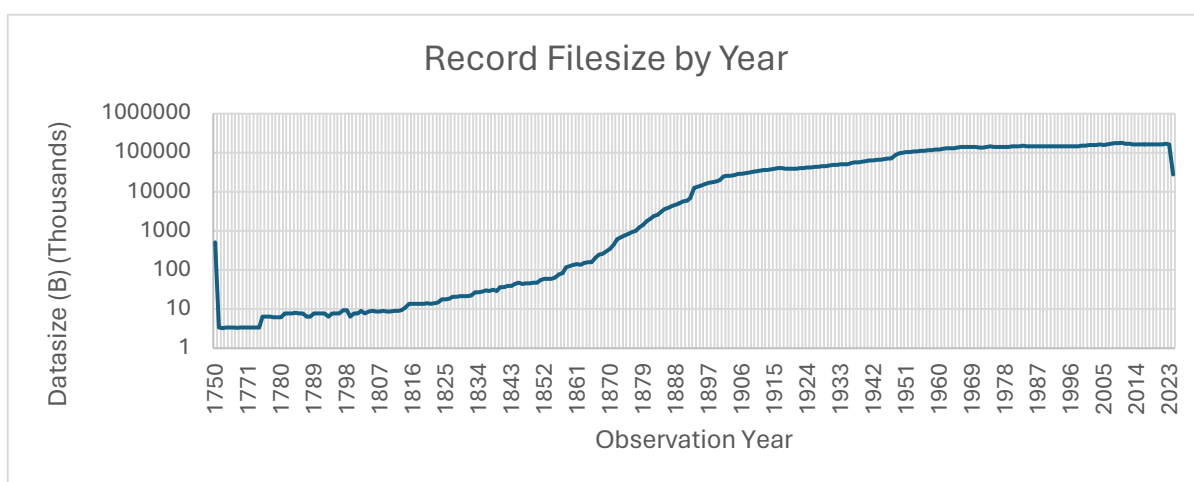


Figure 2 Record Filesize By Year Graph

The footprint of daily records has increased over time per above; we can't yet say whether number of stations or perhaps the frequency of reporting has changed. The file size was obtained with HDFS dfs -du -h to obtain the on-disk file size.

/data/ghcnd/..	Size on cluster (B)	Human Readable Units
ghcnd-countries.txt	3,659	3,659B
ghcnd-inventory.txt	34,380,032	32.81 MB
ghcnd-states.txt	1,086	1086B
ghcnd-stations.txt	10,834,968	10.34 MB
/daily	13,278,940,368	12.36 GB

Schemas

For the scope of the background processing, that is processing that is not involving significant numbers of rows we can start a spark executor with only 2 executors; 1 core per executor; and 1gb of memory on each as well as 1gb on the master node.

As the schema for daily is given explicitly in the GHCND readme; noting that the OBSERVATION TIME and DATE columns are loaded as IntegerType() and not DateType() or StringType(). We can determine, for example, the year using floor division instead of converting the number to a string. This also simplifies comparisons, making it easier to test if one date is larger than the other.

```

schema = StructType([
    StructField("ID", StringType(), False),
    StructField("DATE", IntegerType(), False),
    StructField("ELEMENT", StringType(), True),
    StructField("VALUE", DoubleType(), True),
    StructField("MEASUREMENT_FLAG", StringType(), True),
    StructField("QUALITY_FLAG", StringType(), True),
    StructField("SOURCE_FLAG", StringType(), True),
    StructField("OBSERVATION_TIME", StringType(), True)
])
  
```

Figure 3 Schema For /daily

All fields were set as nullable except ID and DATE as intuitively data missing these attributes is essentially useless in the context of this analysis.

The schemas for all tables are available in Supplementary Materials.

1000 Rows of daily was loaded and cached as a new RDD for use further down; note that on subsequent 'run throughs' of the jupyter notebook; the cache was removed, and the dependant logic commented out.

The files not contained within /daily required a schema to be defined to provide column names; this is still based on the rough schema given by GHCND, however the columns of Countries, Sates and Inventory are defined based on splitting each fixed width row rather than delimiters or a header.

Each table was loaded and .count() called to calculate the number of rows; .count() is used extensively in this analysis and it doesn't carry any major overhead (i.e. local count on each executor, then aggregation to the master node)

Table	Number of rows
Countries	219
Inventory	747382
States	74
Stations	125983
Stations Not Containing WMO_ID	118023

Something to note is that 74 states are provided; the file includes rows for Canadian states.

The status of rows containing WMO_ID was simply filtered for null values in the WMO_ID column.

Transforming Metadata Tables

The GHCND data will requires some transformation to provide greater detail. Intermediary exports are saved to 'hdfs:///user/nki38/outputs/ghcnd'.

Note: at this stage we are 'beginning' our transformations with the RDD's loaded onto our executors as they are when they are first read from their corresponding .csv files.

Firstly, we augment the initial RDD (create RDD '2') using sparks inbuilt substring parser to create a column of the two-letter FIPS country code from each ID.

That is, before and after:

[ID, LAT,LON,ELEVATION, STATE,NAME, GSN, HCN/CRN, WMO_D
[ID, LAT,LON,ELEVATION, STATE,NAME, GSN, HCN/CRN, WMO_D, COUNTRY NAME]

This now means that we can left join the new RDD onto the countries RDD using a standard spark left join (as the number of rows here is not large).

Following about the same logic as above; we left join stations on states, noting that state codes are only available for stations in the US; that is using a customer join condition that matches against the 'US' literal string. That is, matching State, on Code and Country against the literal string 'US'.

The resulting DFs contains useful extra information about the locations of stations.

Inventory

We can make some interesting observations about inventory.csv; As mentioned earlier there are 5 core elements that these stations observe; that is PRCP,TMAX,TMIN,SNWD and SNOW

For precipitation, maximum and minimum observed temperatures, snow depth and snowfall, all on single days defined in the DATE column in daily. Inventory records the breadth of observations for weather types.

The data in inventory.csv is structured to show the period for which some element is observed; we can gain some insight into the age of and whether a station is still recording observations by defining an aggregation step which calculates the minimum and maximum year in which observations were recorded for each station.

For clarification, in the context of an RDD map or reduce function being called throughout this report, it is meant that the step is added to the DAG representing the overall job. How spark organizes and optimises these steps is treated as a black box; further, if it is not otherwise stated, a result has been created as without implied dependencies. There are instances where downstream RDD dependencies may occur, but for most of the results, essentially that job finished with just that output. (Purely in the interest of streamlining development). All steps are expressed in the context from being members of the DAG.

For this report, steps described as follows are abstractions of the generation of the spark DAG execution pipeline.

Firstly, the group By function for the 'ID' column is called; which groups locally on each executor.

We then aggregate; which may require shuffling the data across nodes depending on how spark has partitioned the data across data frames; this is preceded by local aggregation (that is continuing to compute the max and min using the inbuilt functions) and eventuates with a new data frame containing the overall highest and lowest value of year per station.

That is, the first and last year that each station was active and collected any element:

Station Id	Earliest Record	Latest Record
ASN00035202	1907	2004
ASN00009850	1989	2023
...	...	...

We can also determine the number of unique elements which each station has collected by summing through a groupby then aggregate function; Specifically. Counting the unique occurrences of each element per station ID; creating new rows for 'num_core_elements) if the element is a member of `core_elements = ['PRCP','SNOW','SNWD','TMAX','TMIN']` and also doing the same if PRCP is a member of that list; summing on the values of these new rows (through the agg function) yield an RDD of the following form:

ID	Number of Elements	Number of Core Elements	Number of Precipitation Elements
US1NCWK0069	5	3	1
US1CTMD0034	5	3	1
US1COME0173	7	3	1
...	...	...	...

The number of stations collecting only precipitation data is determinable simply by filtering the RDD for rows where 'Number of Precipitation Elements' and 'Number of Elements' are the same. Similarly, filtering for the row: Number of Core Elements having size 5 yields the following result:

Stations collecting all 5 elements	20467
Stations collecting only Rainfall	16301

Later, we will explore records regarding the 'core' elements. For now we can observe the number of number of stations who collected all five of the core elements.

While we could have used collect-set to determine a set of all elements on each station, then joined that back onto stations; this also yields a similar result (and the compute time is shorter than the time it takes the spark executor to

ready the job). The shortcoming of this choice is that the resultant RDD does not contain information about the actual composition of observations except what is in the table(s) above.

Enriched Stations Table

We have created an enriched version of the stations table above; while we could cache the RDD; it is prudent to save the output to disk (we will use a larger mix of executors and memory later and loading directly from a .csv file will reduce overall complexity)

The optimal choice here was to save as a parquet file; however for the sake of portability and human readability (as well as my own familiarity), the file was written to csv; noting that there was also no syntax conflicts in the columns of the augmented csv as the logic producing the RDD only includes counts of elements and not, for example, a list of them.

If the augmented stations_enriched RDD were much larger, parquet would provide an improvement. Similarly; for the sake of human readability; The RDD for augmented stations was repartitioned into a single partition because Spark's write to csv function writes one file per partition; Repartitioning creates shuffle overhead, but again this is negligible in this context.

Merging with Daily

Following the stations_enriched RDD, we discover that there are a number of stations who did not report in the cached 1000 row sample from the schemas section.

We can tell that there are 3 stations who do appear in daily but not in the metadata table for stations or, therefore, in the augmented table. We obtain this by simply joining new stations on daily on the ID field and filtering for rows who have NaN or null values in nullable fields; i.e. latitude.

While this may provide false positives as we aren't dropping rows who have nulls in these places; it is sufficient as the 3 stations can be trivially checked and confirmed to fit the above with filtering.

Were we to remove the limiter, it becomes impractically expensive to join all of daily and stations_augmented,

Performing a full left join between the entire daily dataset and the stations metadata table could be quite resource-intensive, particularly as our dataset grows in size. This approach entails significant shuffling of data across the cluster, resulting in substantial network overhead and potentially prolonged processing times. As our dataset expands, these scalability concerns become increasingly pronounced.

To determine if any stations in the daily data don't appear in the stations metadata table without resorting to a left join we could use filtering and broadcasting joining, as above. By filtering the station IDs from both datasets and broadcasting the smaller station IDs dataset, we can efficiently pinpoint unmatched stations without the need for a full join operation.

This approach handles memory constraints well; and therefore reduces computational load as there's no need to create another stage after a shuffle (as we will not require one).

In all cases; a left anti join will generate an RDD containing only the rows who are unmatched; which is our set of all rows who do not appear in the metadata table.

Analysis

Affiliations of stations

We can make some interesting observations about stations using our Augmented Stations table from earlier; this time pulling in the .csv file from the output directory.

While the header is inferred; we still define a similar schema per the previous section We can apply some trivial filters to obtain the following information and call .count() on each query.

	Count of stations with Flag
Total Stations	125983

Total Active in 2024	31837
GSN	991
HCN	1218
CRN	234
Members of > 1 Network	15
Total in Southern Hemisphere	25316
Stations in New Zealand	15

Geographical Distance(s)

To determine the geographical distance between points given latitude and longitude the haversine distance^[1] as a spark user defined function; A point to note was that it was at this point that choosing the initial schema to be nullable for stations has proven problematic; as such the udf simply handles the case where any member of the two latitudes and longitudes are None and returns 0 in lieu of dropping null rows in the RDD itself.

Scoping to rows for NZ stations was achieved by filtering on the first two characters of ID; the data is then cross joined with a join condition that ensures that we don't create duplicate rows (by setting the join condition to be a '<' comparison such that matchings only occur in one direction.) A new column is then created using the haversine distance UDF which was registered above using the two latitude and longitude values. The Data Frame is ordered by the new 'Distance_km' column which yields the following first row:

NZ000093417	-40.900002	174.983002	PARAPARAUMU AWS	
+				
NZM00093439	-41.333000	174.800003	WELLINGTON AERO AWS	52.270061

These are, therefore the two geographically closest stations in New Zealand given by the stations metadata table. (and this is intuitively correct given the proximity of wellington to paraparaumu)

The table of all unique combinations of stations within New Zealand is saved to the output directory as a .csv file with a header; the file is trivially small so human readability was prioritized over other formats. The same delimiter comma scheme was used with newline breaks.

Something to be wary of is the kind of implementation; for the purpose of processing, each of the following headings were made uniquely. Later we will filter for all new Zealand stations again.

3a

The block size which the HDFS instance(s) have been allocated is unchanged from the 134217728B (128MB) default. The entirety of /daily is of size:

/	Size(B)	Size(GB (2dp))
Compressed	13,278,940,368	13.28
Uncompressed	106,231,522,944	106.23

Noting that !hdfs -du -s returns values for compressed and uncompressed files given that all files in /daily are of type .gz archive. This can be validated by examining the block size of /daily with fsck; noting an average block size of 41238945B (41.23MB) per each of 322 blocks, yielding the same approximate 13.28GB. Further, the years 2023 and 2024 have the following blocks and sizes:

File	On disk size MB	Blocks	Average Size B	Average Size MB
2024.csv.gz	27.49	1	27492832	27.49
2023.csv.gz	166.37	2	83183744	83.18
1900.csv.gz	19.51	1	19509289	19.51

Table 1 - File Sizes on disk with MB simplified

Note that there is significant variance in our block sizes; this is in the interest of ‘fitting’ the files best to each block. 1900.csv.gz and 2024.csv.gz illustrate this as they simply fit into a single block. 2023.csv.gz is too large to be contained within a single 128b block so it is simply distributed by HDFS as per the HDFS implementation.

Total blocks (validated): 2 (avg. block size 83183744 B) Noting that
 Note stage1 has [task1... task2... task n] ... Stage2 has [task 1... task2 ... taskn]
 Stages are separated by shuffles.

Daily Observations and Tasks

Intuitively, an RDD operation like rdd.count() should yield two stages; one task in the final stage and a number of tasks in the first stage that is equal to the number of executor tasks for that stage. For 2023.csv.gz that would be two as it occupies two blocks and for 2024.csv.gz that would be one. Therefore, we would expect three and two tasks respectively.

Year (.csv.gz)	Num Partitions	Num Observations	Num Tasks	Num Stages
2023	1	37,395,852	2	2
2024	1	6, 061,827	2	2

This is not the case; As .gz files require the entirety of the file to be available to uncompress consistently, Spark moves the compressed files onto a sequence of blocks on a single executor such that that spark executor can conduct the extraction locally into a single partition of an RDD, maintaining that the compressed file is not split.

We can see that in explicit detail by loading the 11 files from 2014 to 2024 inclusive in /ghcnd:

Year (.csv.gz)	Number of Observations	Number of Tasks	Number of Stages
2014-2024	375,480,892	105	2

While the structure of /daily does mitigate some of the benefits given by parallelization, the scope of this report is not computationally demanding enough that this would be prohibitive, so no data is transformed for the purpose of fitting it to smaller partitions. By decreasing the archive size, we can store more per partition on each executor; note that there are already small archives (as we saw above where the size of /daily/ records increases greatly over time and therefore we can parallelize more of the overall compute as smaller, more numerous partitions are created.

A way in which we could re distribute the data rows is to divide the .csv into subdirectories of months, i.e. /daily/2021/JAN.../daily/2021/DEC, which is useful from a human-readable point of view and naturally in the case where it is impractical to load all rows from files in the /daily/2021 directory (for example). However, choices about this must consider the volume of entries, i.e. if some n% of entries occur in December, and that n is significantly more than one 1/12th of the number of rows, then perhaps we could consider numbering as entries to n and decide some n such that each part fits into one block, forgoing the real-world attribute in favour of equal distribution.

As such, spark does and would be able to combine small files into larger partitions, in fact, we see that happening with many of the early year records.

Alternatively, while maintaining the existing file structure, an option is to load from /daily/ then immediately re-partition. This does create overhead through a full shuffle; however, depending on the scope of compute, this may be a prudent step. The repartitioned rdd can be cached and used for other stages and tasks. The advantage here is that this can happen in memory, or as close to in memory as spark allows and forgoes the time overhead that significant size differences of partitions may create between executors; where one has a large partition with an upstream dependency which causes ‘done’ executors to sit idle until it’s compute is complete.

Total Observations

For the sake of keeping things together; cells in the Analysis notebook were just run individually so the overall DAG for each of the following tasks doesn’t include extraneous stages.

Total Observations in Daily	31,039,54,141
Total Observations of Core Elements	1,562,456,259

Total Number of Observations in NZ	485521
Total Number of rows in Exported NZ Observations	485521

Core Element Counts

Over the entirety of /daily the observations of core elements are of the following distribution:

Element	Count of observations over /daily
TMIN	454,759,421
TMAX	455,946,095
PRCP	1,069,105,193
SNOW	353,904,309
SNWD	297,846,434

Rainfall, as PRCP is the most observed element in the core elements.

These values were obtained over all daily by firstly filtering the RDD for all daily rows for rows whose 'ELEMENT' value appears in the list of core elements; then grouping by element and reducing by aggregating on those counts.

Temperature Matching

An interesting attribute of the GHCND dataset is temperature reporting. As mentioned in the background section, two of the core reporting values are the maximum observed temperature on a day TMAX and the minimum TMIN. While we would expect that these values are recorded in pairs, this is not the case.

A useful insight is therefore matchings of TMIN and TMAX; that is average temperatures are of not particularly useful if there are significantly more of one observation present

While in testing a .limit(1000) was applied to the all_daily. For a dataset of that size simply performing a broadcast left anti join on daily with the result of filtering for TMAX is quick, filtering for TMAX from all of daily yields a dataframe which does not fit within memory on a single worker node and therefore the broadcast join requires expensive shuffling.

Firstly, the RDD all_daily is filtered for TMIN and TMAX; then only the relevant columns are selected (ID, DATE, ELEMENT and Value) for the sake of memory footprint reduction when we move to the next two steps.

The resultant RDD is grouped by ID and Date and Elements are aggregated such that we now have ID, DATE and an array of all elements; from here we simply filter out rows which are not solely TMIN.

Total TMIN stations	9,322,723
Total TMIN or TMAX stations	91,070,5516

The cursory outline of this job is as follows:

Step	Type	Stage	Action
1	Read		Read_csv with schema into DF – Partitions on executor
2	Map	1	Filter for TMIN & TMAX
3	Map	1	Select [ID, DATE, ELEMENT,VALUE]
4	Reduce	2	Count Results
5	Map	2	Group By ID & Date locally from RDD in 3
6	Reduce	3	Aggregate on Set of Element Values
7	Map	3	Filter Elements contains only TMIN
8	Reduce	4	Take .count() for number of TMIN

Jobs are the rows between each change in stages that are not reductions.

Temperatures in New Zealand

We can obtain the set of all observations within New Zealand stations by filtering Daily for the first two characters of country code similar to how we did for inventory earlier.

Total Number of Stations in NZ	485520
Lowest Year Observed	1940
Greatest Year Observed	2024

Note, that while we have observed only the YYYY component of the date; the aggregation was performed to calculate the max and min of the Date columns without extracting the year. There was no performance difference in adding another column and job to this stage of processing; we can see an 84y reporting period for stations in NZ.

And we can conveniently write this back to the host filesystem for our spark instance with `!hdfs dfs copyToLocal *`. Where `*` copies all of our previous written to disk outputs back to the local fs. Verifying with `!wc -l` on the `NZ_observations.csv` export, in fact the number of rows is consistent with the RDD.

Average Temperatures in NZ as reported by GHCND:

In order to visualize this data, we could simply pass the entire dataframe into matplotlib (or similar) however; the density of datapoints is far too great to infer useful information from. To mitigate this we calculate a rolling average using a window function on the new Zealand stations dataframe and take an nth sample.

Sample TMIN vs TMAX graphs for two of fifteen stations.

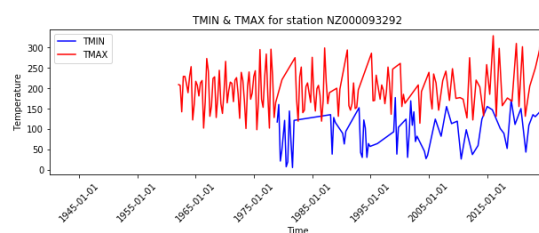


Figure 4 TMIN TMAX comparison for Sample Station 1

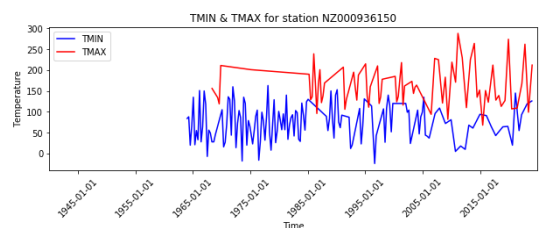


Figure 5 TMIN TMAX comparison for Sample Station 2

We can also observe the total rainfall observed across all stations for a given month, again the other 11 are in the working directory. To obtain this we again filter by TMIN TMAX on the initial RDD from read; This time grouping by month after casting the string to spark date type. Then grouping on Year Month and Element, aggregating while calculating the average across these, then reformatting the output to push into matplotlib^[2]

Regarding the graphs; it is clear that there are missing data points (i.e. the long linear lines we see between points like 1965-1975 in the graph on the right). Realistically; these should not appear as lines. We could either change the filtering to take non-linear parts of the data; instead clustering around time, or changing the matplotlib inputs such that spaces over some threshold are not graphed. Whether the latter is possible, I am unsure.

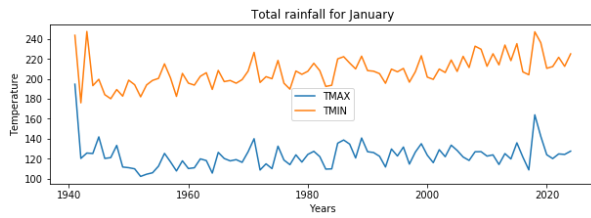


Figure 6 Rainfall NZ for sample month

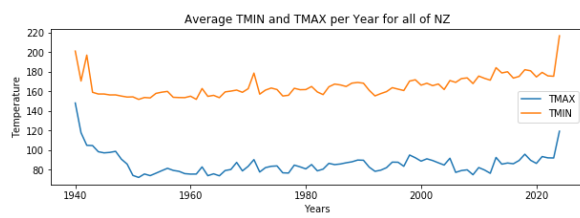


Figure 7 TMIN TMAX Delta for NZ

Finally; we can plot rainfall over all of New Zealand in a similar way (above)

Average Rainfall Worldwide

We can transform the .daily RDD to obtain the average Rainfall by country. We can do this by grouping on ID, DATE. One of the most pushing challenges here is the sheer file size:

Total number of rows after filtering PRCP for 2023	17541
--	-------

Therefore, the order of operations is particularly important; as there must be an aggregation step. After performing the same typecasts as above; each partition is grouped locally on its executor then grouped through the aggregate reduce step; calculating the floating point average of value at each step. The resulting RDD yields three columns; YEAR, COUNTRYCODE and VALUE.

To obtain the country with the greatest rainfall, we aggregate again; this time for max value.

Country with most rainfall:	Equatorial Guinea	4361.0mm p/a average
-----------------------------	-------------------	----------------------

Finally, the average precipitation over 2023 for the planet is graphed using a choropleth; Note it was at a point well after executing the job and generating the choropleth's that I have realised that I've simply taken the average rainfall per observation, not per year.

Rainfall MM 2023

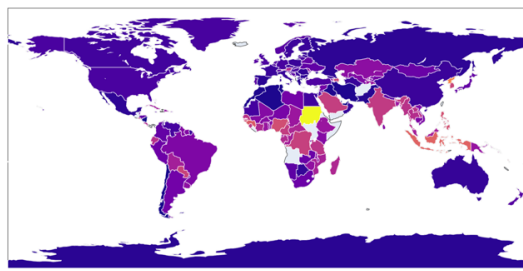


Figure 8 precipitation by Country

Rainfall MM 2023

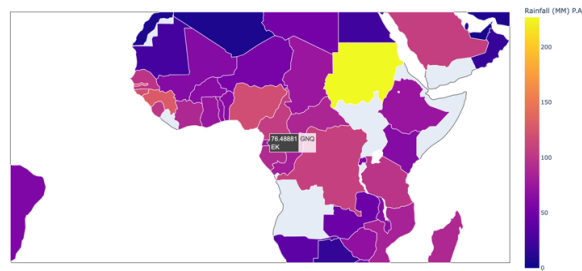


Figure 9 Precipitation Zoom

Comparing to earlier; Equatorial Guinea, who recorded the greatest overall rainfall currently sits below the average annual rainfall of 5364 units across 2023. Geographically; the closeness of these regions is consistent.

Sudan reported the greatest rainfall per this analysis; however a cursory google indicates that this is very likely not the case (i.e that their rainfall is much less than the typical median rainfall). This is probably due to some outlier data and could be fixed by pre-processing the RDD.

Note the few countries who do not report GHCND data are represented as blank spaces on the RDD.

Conclusions:

There are over 31 billion observations in /daily however the metadata tables are trivially small. We were able to process and augment the metadata tables in a slightly reckless way due to the negligible real world compute time. However, the volume of daily doesn't afford that.

Therefore; within the constraints of spark we devised close to as efficient as possible implementations for filtering and organizing the daily to compare and contrast rainfall and temperature deltas overtime. The full series for these is

available separately.

We can observe average temperature trends in New Zealand directly through the graph outputs and also global rainfall.

A useful next step would be to generate more of these graphs; perhaps one per year for the choropleth; that way a time delta can be used to observe changes in precipitation.

Notable challenges in processing were.

1. Maintaining a 'clean' DAG will working within Jupyter notebooks; that is when some points of interest are already computed but they'd need to be removed before running the notebook environment again to prevent spark from adding them to the DAG.
2. While I believe every question was answered per the guidelines; ambiguity around states and some wordings meant that I took a swing in the dark.
3. I used a google AI generated list of American territories for a chunk of development and that caused mismatches in data and nonsensical results.
4. I have hopefully included at least one of every relevant explanation in this document.
5. **I cannot (as at the 2nd of may) begin a Spark session, so the prcp over all countries field is blank but the logic to obtain the value is implemented in the notebook.**

Supplementary material

4x Jupyter Notebooks are provided; Analysis, Processing, Graphing and Graphing World.

1x pdf for graphs for months of rainfall in NZ.

1x pdf for rainfall by station in NZ : TMIN TMAX

1x pdf with additional Choropleth for the United States.

Acknowledgements and References:

[1] Haversine Distance disclosure: This function was partially generated using ChatGPT 3.5 on April 3rd, 2024, through Microsoft Co-pilot

[2] Much of the code used for generating the matplotlib plots was generated and tweaked; at least boilerplate was created with generative tools.

The entirety of this document was written organically and is entirely my own work, however, M365 provides some grammatical correction tools and those were leveraged.